

NLA Challenge 2

Generated by Doxygen 1.15.0

1 Clustering - Numerical Linear Algebra Challenge 2	1
1.1 Authors	1
2 Namespace Index	3
2.1 Namespace List	3
3 File Index	5
3.1 File List	5
4 Namespace Documentation	7
4.1 Constants Namespace Reference	7
4.1.1 Detailed Description	7
4.2 LisIO Namespace Reference	7
4.2.1 Detailed Description	7
4.2.2 Function Documentation	8
4.2.2.1 load_eigenpairs()	8
4.2.2.2 load_matrix()	8
4.2.2.3 load_vector()	8
4.2.2.4 save_matrix()	9
4.2.2.5 save_vector()	9
4.3 MtxIO Namespace Reference	10
4.3.1 Detailed Description	10
4.3.2 Function Documentation	10
4.3.2.1 load_matrix()	10
4.3.2.2 load_vector()	10
4.3.2.3 save_matrix()	11
4.3.2.4 save_vector()	11
4.4 Types Namespace Reference	11
4.4.1 Detailed Description	11
5 File Documentation	13
5.1 matrix_io.hpp	13
5.2 matrix_ops.hpp	13
5.3 types.hpp	14
5.4 src/matrix_io.cpp File Reference	14
5.5 src/matrix_ops.cpp File Reference	14
5.5.1 Function Documentation	15
5.5.1.1 compute_permutation_matrix()	15
5.5.1.2 frobenius_norm()	15
5.5.1.3 is_positive_definite()	15
5.5.1.4 is_semi_positive_definite()	16
5.5.1.5 is_symmetric()	16
5.5.1.6 make_diag_matrix()	16
5.5.1.7 make_laplacian()	17

5.5.1.8 <code>order_eigenvector()</code>	17
5.5.1.9 <code>row_sums()</code>	17

Index	19
--------------	-----------

Chapter 1

Clustering - Numerical Linear Algebra Challenge 2

This project explores spectral clustering techniques applied to a social network graph, using numerical linear algebra tools. The main objective is to identify communities within a network by leveraging the Fiedler vector, i.e., the eigenvector corresponding to the second smallest eigenvalue of the graph Laplacian.

1.1 Authors

- Vale Turco - 10809855
- Samuele Allegranza - 10766615
- Valeriia Potrebina - 11114749

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Constants	7
LisIO	7
MtxIO	10
Types	11

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/matrix_io.hpp	13
include/matrix_ops.hpp	13
include/types.hpp	14
src/matrix_io.cpp	14
src/matrix_ops.cpp	14

Chapter 4

Namespace Documentation

4.1 Constants Namespace Reference

Variables

- const std::string **TASK1_GRAPH_FILE** = std::string("./media/matrices/task1_graph.mtx")
- const std::string **SOCIAL_GRAPH_FILE** = std::string("./media/matrices/social.mtx")
- const std::string **SOCIAL_LAPLACIAN_GRAPH_FILE** = std::string("./media/matrices/social_laplacian.mtx")
- const std::string **SOCIAL_EVEC_FILE** = std::string("./media/lisres/task3_evecs_second_smallest.mtx")
- const std::string **SOCIAL_EVAL_FILE** = std::string("./media/lisres/task3_evals_second_smallest.mtx")
- const std::string **SMALL_MTX_FILE** = std::string("./media/matrices/test_mats/small.mtx")
- const std::string **SMALL_EIGEN_VEC_FILE** = std::string("./media/matrices/test_mats/small_vector_eig.mtx")
- const std::string **SMALL_LIS_VEC_FILE** = std::string("./media/matrices/test_mats/small_vector_lis.mtx")
- constexpr Types::Real **eps** = 1.0e-15

4.1.1 Detailed Description

Contains constants used within this library

4.2 LisIO Namespace Reference

Functions

- bool [load_matrix](#) (const std::string &filename, Types::Sparse &matrix)
- bool [load_vector](#) (const std::string &filename, Types::Vector &vector)
- bool [load_eigenpairs](#) (const std::string &vec_filename, const std::string &eval_filename, std::vector<Types::EigenPair> &eigenpairs)
- bool [save_matrix](#) (const std::string &filename, const Types::Sparse &matrix)
- bool [save_vector](#) (const std::string &filename, const Types::Vector &vector)

4.2.1 Detailed Description

Contains functions to load and store mtx files that are immediately readable with LIS

4.2.2 Function Documentation

4.2.2.1 load_eigenpairs()

```
bool LisIO::load_eigenpairs (
    const std::string & evec_filename,
    const std::string & eval_filename,
    std::vector< Types::EigenPair > & eigenpairs)
```

Load two LIS generated mtx file into a vector of EigenPairs

Parameters

<i>evec_filename</i>	The LIS generated mtx file containing the eigenvectors
<i>eval_filename</i>	The LIS generated mtx file containing the eigenvalues
<i>eigenpairs</i>	A vector of EigenPairs to be filled, this will clear previous data

Returns

true if file readings were successful, false otherwise

4.2.2.2 load_matrix()

```
bool LisIO::load_matrix (
    const std::string & filename,
    Types::Sparse & matrix)
```

Load a LIS generated mtx file into a Eigen Sparse matrix

Parameters

<i>filename</i>	The LIS generated mtx file containing the sparse matrix
<i>matrix</i>	The matrix to be filled, this will clear previous data

Returns

true if file reading was successful, false otherwise

4.2.2.3 load_vector()

```
bool LisIO::load_vector (
    const std::string & filename,
    Types::Vector & vector)
```

Load a LIS generated mtx file into a Eigen VectorXd

Parameters

<i>filename</i>	The LIS generated mtx file containing the vector
<i>vector</i>	The vector to be filled, this will clear previous data

Returns

true if file reading was successful, false otherwise

4.2.2.4 save_matrix()

```
bool LisIO::save_matrix (
    const std::string & filename,
    const Types::Sparse & matrix)
```

Save a Eigen Sparse into a file immediately readable from LIS

Parameters

<i>filename</i>	The mtx file to be filled with LIS readable data, this will delete previous content
<i>matrix</i>	The matrix to be saved

Returns

true if file writing was successful, false otherwise

4.2.2.5 save_vector()

```
bool LisIO::save_vector (
    const std::string & filename,
    const Types::Vector & vector)
```

Save a Eigen VectorXd into a file immediately readable from LIS

Parameters

<i>filename</i>	The mtx file to be filled with LIS readable data, this will delete previous content
<i>vector</i>	The vector to be saved

Returns

true if file writing was successful, false otherwise

4.3 MtxIO Namespace Reference

Functions

- bool [load_matrix](#) (const std::string &filename, Types::Sparse &matrix)
- bool [load_vector](#) (const std::string &filename, Types::Vector &vector)
- bool [save_matrix](#) (const std::string &filename, const Types::Sparse &matrix)
- bool [save_vector](#) (const std::string &filename, const Types::Vector &vector)

4.3.1 Detailed Description

Contains functions to load and store mtx files that are immediately readable with Eigen

4.3.2 Function Documentation

4.3.2.1 load_matrix()

```
bool MtxIO::load_matrix (
    const std::string & filename,
    Types::Sparse & matrix)
```

Load matrix in mtx file into an Eigen Sparse

Parameters

<i>filename</i>	The mtx file containing the sparse matrix
<i>matrix</i>	The matrix to be filled, this will clear previous data

Returns

true if file reading was successful, false otherwise

4.3.2.2 load_vector()

```
bool MtxIO::load_vector (
    const std::string & filename,
    Types::Vector & vector)
```

Load vector in mtx file into an Eigen VectorXd

Parameters

<i>filename</i>	The mtx file containing the vector
<i>vector</i>	The vector to be filled, this will clear previous data

Returns

true if file reading was successful, false otherwise

4.3.2.3 save_matrix()

```
bool MtxIO::save_matrix (
    const std::string & filename,
    const Types::Sparse & matrix)
```

Save an Eigen Sparse into mtx file immediately readable by Eigen

Parameters

<i>filename</i>	The mtx file to be filled, this will delete previous content
<i>matrix</i>	The matrix to be saved

Returns

true if file writing was successful, false otherwise

4.3.2.4 save_vector()

```
bool MtxIO::save_vector (
    const std::string & filename,
    const Types::Vector & vector)
```

Save an Eigen VectorXd into a mtx file immediately readable by Eigen

Parameters

<i>filename</i>	The mtx file to be filled, this will delete previous content
<i>vector</i>	The vector to be saved

Returns

true if file writing was successful, false otherwise

4.4 Types Namespace Reference

Typedefs

- using **Real** = double
- using **Index** = Eigen::Index
- using **Sparse** = Eigen::SparseMatrix<Real, Eigen::RowMajor>
- using **Full** = Eigen::MatrixXd
- using **Vector** = Eigen::VectorXd
- using **EigenPair** = std::pair<Vector, Real>
- using **OrderEigenVector** = std::vector<std::pair<Index, Real>>

4.4.1 Detailed Description

Contains types used within this library

A vector to be ordered, such that if element i is (j, value) then value was moved from position j to i

Chapter 5

File Documentation

5.1 matrix_io.hpp

```
00001 #ifndef CHALLENGE2_MATRIX_IO_HPP
00002 #define CHALLENGE2_MATRIX_IO_HPP
00003
00004 #include "types.hpp"
00005 #include <iostream>
00006 #include <unsupported/Eigen/SparseExtra>
00007
00011 namespace MtxIO {
00012     bool load_matrix(const std::string &filename, Types::Sparse &matrix);
00013     bool load_vector(const std::string &filename, Types::Vector &vector);
00014     bool save_matrix(const std::string &filename, const Types::Sparse &matrix);
00015     bool save_vector(const std::string &filename, const Types::Vector &vector);
00016 };
00017
00021 namespace LisIO {
00022     bool load_matrix(const std::string &filename, Types::Sparse &matrix);
00023     bool load_vector(const std::string &filename, Types::Vector &vector);
00024     bool load_eigenpairs(const std::string &vec_filename, const std::string
&eval_filename, std::vector<Types::EigenPair> &eigenpairs);
00025     bool save_matrix(const std::string &filename, const Types::Sparse &matrix);
00026     bool save_vector(const std::string &filename, const Types::Vector &vector);
00027 }
00028
00029 #endif //CHALLENGE2_MATRIX_IO_HPP
```

5.2 matrix_ops.hpp

```
00001 #ifndef CHALLENGE2_MATRIX_OPS_HPP
00002 #define CHALLENGE2_MATRIX_OPS_HPP
00003
00004 #include "types.hpp"
00005
00006 Types::Real frobenius_norm(const Types::Sparse &matrix);
00007 Types::Vector row_sums(const Types::Sparse &matrix);
00008 Types::Sparse make_diag_matrix(const Types::Vector &vector);
00009 Types::Sparse make_laplacian(const Types::Sparse &matrix);
00010
00011 std::vector<Types::EigenPair> compute_eigenpairs(const Types::Sparse &matrix);
00012 bool is_symmetric(const Types::Sparse &matrix);
00013 bool is_positive_definite(const Types::Vector &eigenvals);
00014 bool is_semi_positive_definite(const Types::Vector &eigenvals);
00015
00016 Types::OrderEigenVector order_eigenvector(const Types::Vector &eigenvector);
00017 Types::Sparse compute_permutation_matrix(const Types::OrderEigenVector &order_eigenvector);
00018
00019 #endif //CHALLENGE2_MATRIX_OPS_HPP
```

5.3 types.hpp

```

00001 #ifndef CHALLENGE02_TYPES_AND_DEFS_HPP
00002 #define CHALLENGE02_TYPES_AND_DEFS_HPP
00003
00004
00005 #include <Eigen/Sparse>
00006 #include <Eigen/Dense>
00007
00011 namespace Types {
00012     using Real = double;
00013     using Index = Eigen::Index;
00014     using Sparse = Eigen::SparseMatrix<Real, Eigen::RowMajor>;
00015     using Full = Eigen::MatrixXd;
00016     using Vector = Eigen::VectorXd;
00017     using EigenPair = std::pair<Vector, Real>;
00022     using OrderEigenVector = std::vector<std::pair<Index, Real>;
00023 }
00024
00025 inline std::ostream& operator<<(std::ostream &os, Types::OrderEigenVector &oev) {
00026     for (auto j=0; j<oev.size(); ++j)
00027         os << " ( " << oev[j].first << " , " << j << " ) : " << oev[j].second << "\n";
00028     return os;
00029 }
00030
00034 namespace Constants {
00035
00036     const std::string TASK1_GRAPH_FILE = std::string("./media/matrices/task1_graph.mtx");
00037     const std::string SOCIAL_GRAPH_FILE = std::string("./media/matrices/social.mtx");
00038     const std::string SOCIAL_LAPLACIAN_GRAPH_FILE =
00039         std::string("./media/matrices/social_laplacian.mtx");
00039     const std::string SOCIAL_EVEC_FILE =
00040         std::string("./media/lisres/task3_evecs_second_smallest.mtx");
00040     const std::string SOCIAL_EVAL_FILE =
00041         std::string("./media/lisres/task3_evals_second_smallest.mtx");
00041
00042
00043     const std::string SMALL_MTX_FILE = std::string("./media/matrices/test_mats/small.mtx");
00044     const std::string SMALL_EIGEN_VEC_FILE =
00045         std::string("./media/matrices/test_mats/small_vector_eig.mtx");
00045     const std::string SMALL_LIS_VEC_FILE =
00046         std::string("./media/matrices/test_mats/small_vector_lis.mtx");
00046
00047     constexpr Types::Real eps = 1.0e-15;
00048 }
00049
00050 #endif //CHALLENGE02_TYPES_AND_DEFS_HPP

```

5.4 src/matrix_io.cpp File Reference

```
#include <matrix_io.hpp>
```

5.5 src/matrix_ops.cpp File Reference

```
#include <matrix_ops.hpp>
#include <algorithm>
```

Functions

- Types::Real [frobenius_norm](#) (const Types::Sparse &matrix)
- Types::Vector [row_sums](#) (const Types::Sparse &matrix)
- Types::Sparse [make_diag_matrix](#) (const Types::Vector &vector)
- Types::Sparse [make_laplacian](#) (const Types::Sparse &matrix)
- bool [is_symmetric](#) (const Types::Sparse &matrix)
- bool [is_positive_definite](#) (const Types::Vector &eigenvals)
- bool [is_semi_positive_definite](#) (const Types::Vector &eigenvals)
- Types::OrderEigenVector [order_eigenvector](#) (const Types::Vector &eigenvector)
- Types::Sparse [compute_permutation_matrix](#) (const Types::OrderEigenVector &order_eigenvector)

5.5.1 Function Documentation

5.5.1.1 compute_permutation_matrix()

```
Types::Sparse compute_permutation_matrix (
    const Types::OrderEigenVector & order_eigenvector)
```

Computes the permutation matrix C_π such that $A * C_\pi$ reorders the columns of the matrix A, the order is given by ordered eigen vector

Parameters

<i>order_eigenvector</i>	The eigen vector to deduce the ordering from
--------------------------	--

Returns

The permutation matrix

See also

[The wikipedia page](#)

5.5.1.2 frobenius_norm()

```
Types::Real frobenius_norm (
    const Types::Sparse & matrix)
```

Computes the Frobenius norm of a Sparse

Parameters

<i>matrix</i>	The matrix to compute the norm of
---------------	-----------------------------------

Returns

the Frobenius norm

Note

Use the Eigen norm() method instead

5.5.1.3 is_positive_definite()

```
bool is_positive_definite (
    const Types::Vector & eigenvals)
```

Check if a matrix is Positive Definite given an Eigen VectorXd containing the eigenvalues

Parameters

<i>eigenvals</i>	the eigenvalues to parse
------------------	--------------------------

Returns

true if all eigenvalues are strictly positive

Note

if an eigenvalue is within Constants::eps distance from 0 it is considered to be 0

5.5.1.4 is_semi_positive_definite()

```
bool is_semi_positive_definite (
    const Types::Vector & eigenvals)
```

Check if a matrix is Positive Semi Definite given an Eigen VectorXd containing the eigenvalues

Parameters

<i>eigenvals</i>	the eigenvalues to parse
------------------	--------------------------

Returns

true if all eigenvalues are greater or equal to 0

Note

if an eigenvalue is within Constants::eps distance from 0 it is considered to be 0

5.5.1.5 is_symmetric()

```
bool is_symmetric (
    const Types::Sparse & matrix)
```

Check if a sparse matrix is symmetric

Parameters

<i>matrix</i>	The matrix to check the symmetry of
---------------	-------------------------------------

Returns

true if the matrix is symmetric, false otherwise

5.5.1.6 make_diag_matrix()

```
Types::Sparse make_diag_matrix (
    const Types::Vector & vector)
```

Generate a diagonal sparse matrix D from a vector such that $D_{ii} = v_i \forall i \in \{0, \dots, n-1\}$

Parameters

<i>vector</i>	The vector to compute the diagonal matrix from
---------------	--

Returns

a diagonal matrix as per description

5.5.1.7 make_laplacian()

```
Types::Sparse make_laplacian (
    const Types::Sparse & matrix)
```

Compute the Laplacian of a matrix

Parameters

<i>matrix</i>	The matrix to compute the Laplacian of
---------------	--

Returns

The Laplacian matrix

5.5.1.8 order_eigenvector()

```
Types::OrderEigenVector order_eigenvector (
    const Types::Vector & eigenvector)
```

Orders values in an eigenvector preserving knowledge of previous ordering

Parameters

<i>eigenvector</i>	the eigen vector to be sorted
--------------------	-------------------------------

Returns

a sorted eigenvector

5.5.1.9 row_sums()

```
Types::Vector row_sums (
    const Types::Sparse & matrix)
```

Computes a vector v from a matrix A such that $v_i = \sum_{j=0}^{m-1} A_{ij} \forall i \in \{0, \dots, n-1\}$

Parameters

<i>matrix</i>	The matrix to compute the vector from
---------------	---------------------------------------

Returns

a vector as per description

Index

Clustering - Numerical Linear Algebra Challenge 2, [1](#)
compute_permutation_matrix
 [matrix_ops.cpp, 15](#)
Constants, [7](#)

frobenius_norm
 [matrix_ops.cpp, 15](#)

include/matrix_io.hpp, [13](#)
include/matrix_ops.hpp, [13](#)
include/types.hpp, [14](#)
is_positive_definite
 [matrix_ops.cpp, 15](#)
is_semi_positive_definite
 [matrix_ops.cpp, 16](#)
is_symmetric
 [matrix_ops.cpp, 16](#)

LisIO, [7](#)
 load_eigenpairs, [8](#)
 load_matrix, [8](#)
 load_vector, [8](#)
 save_matrix, [9](#)
 save_vector, [9](#)
load_eigenpairs
 [LisIO, 8](#)
load_matrix
 [LisIO, 8](#)
 [MtxIO, 10](#)
load_vector
 [LisIO, 8](#)
 [MtxIO, 10](#)

make_diag_matrix
 [matrix_ops.cpp, 16](#)
make_laplacian
 [matrix_ops.cpp, 17](#)
matrix_ops.cpp
 compute_permutation_matrix, [15](#)
 frobenius_norm, [15](#)
 is_positive_definite, [15](#)
 is_semi_positive_definite, [16](#)
 is_symmetric, [16](#)
 make_diag_matrix, [16](#)
 make_laplacian, [17](#)
 order_eigenvector, [17](#)
 row_sums, [17](#)
MtxIO, [10](#)
 load_matrix, [10](#)
 load_vector, [10](#)
 save_matrix, [10](#)
 save_vector, [11](#)

order_eigenvector
 [matrix_ops.cpp, 17](#)

row_sums
 [matrix_ops.cpp, 17](#)

save_matrix
 [LisIO, 9](#)
 [MtxIO, 10](#)
save_vector
 [LisIO, 9](#)
 [MtxIO, 11](#)
src/matrix_io.cpp, [14](#)
src/matrix_ops.cpp, [14](#)

Types, [11](#)